Clustering

Clustering

Given a set of samples find a partition into a finite number of clusters so that samples within each cluster are more similar to each other than to those in different clusters

- Grouping patients with similar physiological or clinical characteristics
- Identifying subgroups of patients within the same disease
- Partitioning medical images into regions with similar characteristics
- Extracting patterns from large electronic health records
- Identifying patterns in physiological signals
- Identifying different cell populations in high-dimensional single-cell datasets
- Identifying groups of genes with similar expression patterns across samples
- Grouping proteins based on structural or sequence similarity
- …

The problems that we need to solve

- How do we measure similarity among samples ?
- How do we find groups of similar samples ?
- How do we define the number of groups ?

Clustering principles

- Compactness-based: a cluster is a set of samples that are close to each other according to a metric
  - K-means
- Model-based: a cluster is a subset of sample generated by an underlying probability distribution
  - Gaussian Mixture Models
- Connectivity-based: a clusters is a set of highly connected points in a graph, where edges represent similarity
  - Hierarchical clustering
- Density-based: clusters are dense region of data space separated by low-density regions
  - DBSCAN

K-means

- Initialization is important
- The algorithm converges to a local minimum

1. Define the number of clusters (k)
2. Generate centers for the k clusters, for example by selecting k random points
3. Assign each sample to the closer cluster center (distance is computed with metric L2)
4. Calculate the cluster centers
5. Go back to 3 until convergence (cluster assignment stops to change) or maximum number of iterations

- Feature scaling is critical. Without normalization, features with large magnitudes dominate the distance computation
- Standardization to zero mean and unit variance is generally recommended before applying distance-based clustering

K-means++ is a modified version of this method that doesnt use random points as starting center but close ones

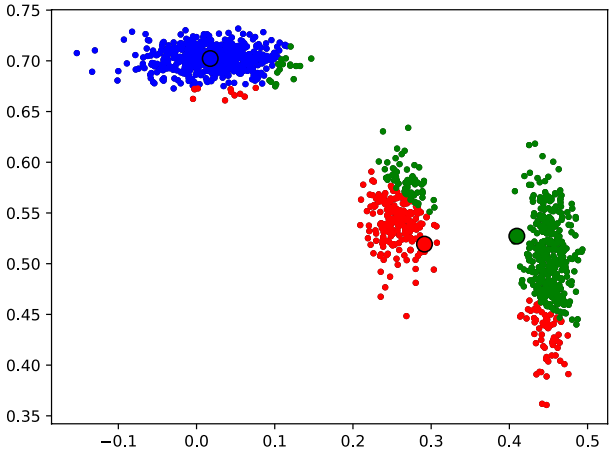
$$J = \sum_{i=0}^{m-1} \left(x^i - C\left(x^i\right) \right)^2$$

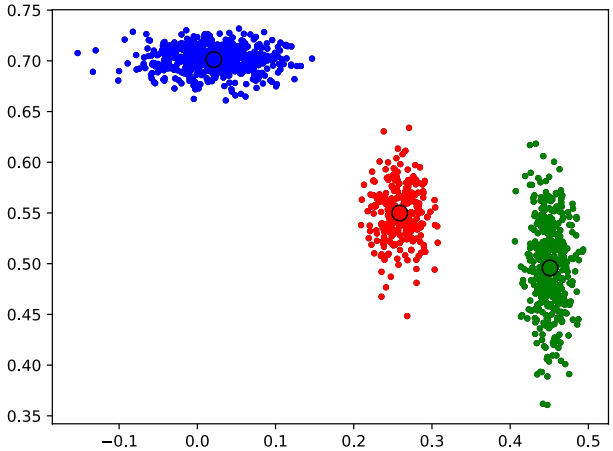
Sum over all the samples

i-th sample

center of the cluster where the i-th sample belongs

To choose the best K we compute the value of the function with each K and choose the first one with an acceptable drop
The J function follows an exponential decay -> Elbow method





K-means

Works well on spherical clusters -> Extreme case of concentrical spherical clusters

- It partitions data into K clusters by minimizing within-cluster variance
- Objective function: minimize the sum of squared distances between data points and their assigned cluster centroids
- It works well in the presence of:
  - spherical clusters
  - clusters of similar sizes
  - clusters with similar densities
- Main limitations are:
  - K needs to be defined in advance
  - detections of non-convex clusters
  - sensitivity to outliers

Kmeans++

It uses a different initialization process that improve convergence

- Choose the first centre randomly
- Compute the distance from each other point to its closest centre
- Choose a new centre with probability proportional to the minimum distance from the other centres

Gaussian Mixture Models GMM

We want to estimate the probability distribution behind the samples

The hypothesis is that samples are generated from a mixture of K gaussian distributions (each with probability π_k) with mean μ_k and covariance matrix $\sum_k$

- The parameters π_k, μ_k, and $\sum_k$ are identified by Expectation Maximization algorithm
- It's similar to K-means but
  - clusters might have non-spherical shapes
  - clusters might have different sizes
- For each sample it provides a soft-clustering probability
- The model can be used to generate new samples

$$p(x) = \sum_{k=0}^{K-1} \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$$

centre of component k

covariance of component k

Normal distribution

weight of the component (cluster) k

$$\mathcal{N}(x|\mu_k, \Sigma_k) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma_k)}} \exp\left(-\frac{1}{2}(x-\mu_k)^T {\Sigma_k}^{-1}(x-\mu_k)\right)$$

$$\sum_{k=0}^{K-1} \pi_k = 1$$ weights are normalized

$$(\boldsymbol{B})\ P(k|x) = \frac{\pi_k \mathcal{N}(x|\mu_k, \Sigma_k)}{\sum_{j=0}^{K-1} \pi_j \mathcal{N}(x|\mu_j, \Sigma_j)}$$

probability that sample x belongs to cluster k

Expectation Maximization algorithm

1. Initialize the parameters randomly (K-means centroid can be used for μ_k)
2. for each point and cluster compute the probability that the point belongs to the cluster (formula B of previous slide)
3. Update means and covariances

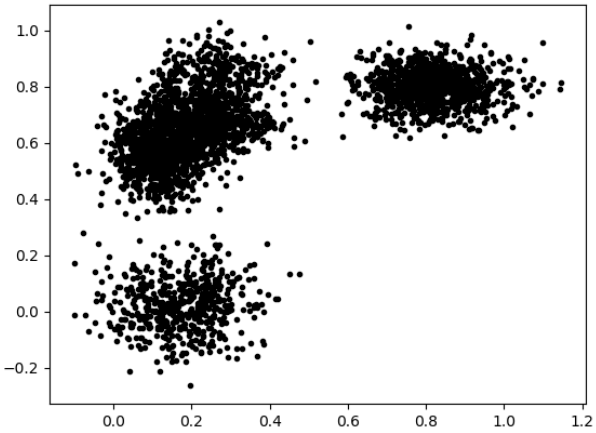
number of samples in cluster k

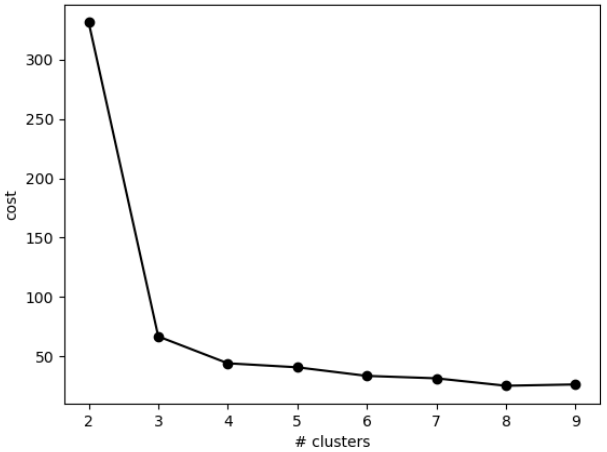
$$N_k = \sum_{i=0}^{m-1} P(k|x_i) \qquad \mu_k = \frac{1}{N_k} \sum_{i=0}^{m-1} P(k|x_i) x_i \qquad \Sigma_k = \frac{1}{N_k} \sum_{i=0}^{m-1} P(k|x_i)(x_i - \mu_k)(x_i - \mu_k)^T \qquad \pi_k = \frac{N_k}{m}$$

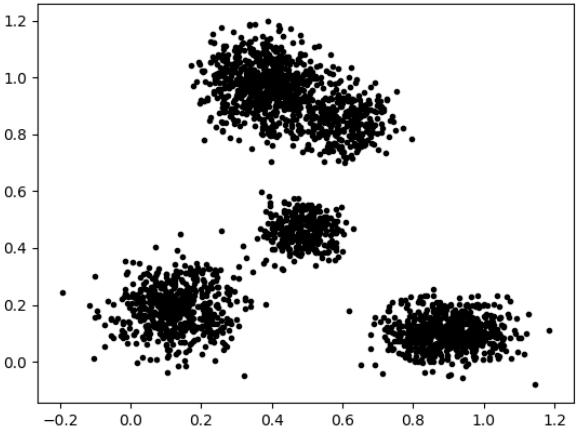
4. Go back to 2 until convergence

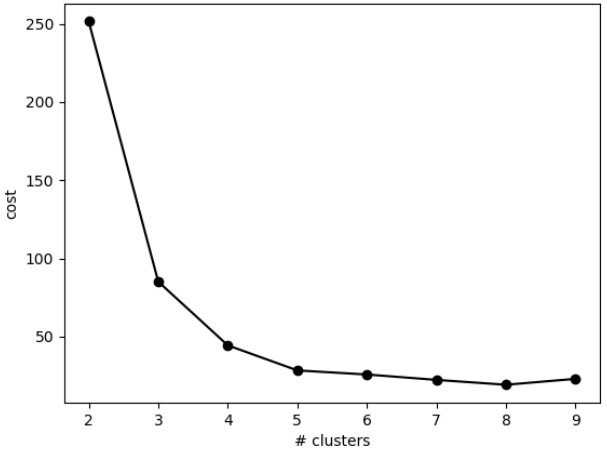
How to define the number of clusters

ELBOW METHOD









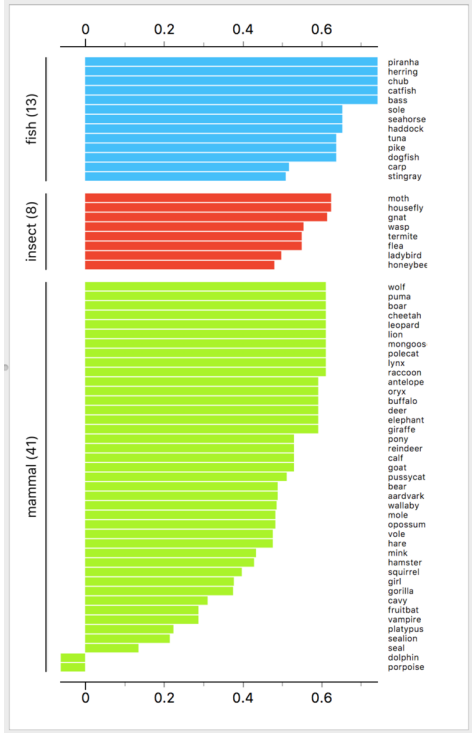
Silhouette Score

- For each point compute:
  - a(i) = the average distance to points in the same cluster
  - b(i) = average distance from the closest cluster

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Silhouette coefficient

- it ranges from -1 to +1
- close to 1 → well-clustered
- near 0 → clusters are overlapping
- close to -1 → likely misclassified



- Compute the average silhouette score for different value for each value of K
- Choose the value that maximize it

→ highest clustering quality

- It evaluates both cluster compactness and separation, while Elbow focuses on compacteness
- Easier to find a maximum that a change in shape as in Elbow method
- Higher computational cost that the Elbow method because of the pair-distances

Akaike Information Criterion (AIC)

It is used to compare and select models by balancing goodness-of-fit against model complexity

$$AIC = 2k - 2\ln(L)$$

number of parameters of the model: this terms penalizes complex models with a high number of parameters

maximum likelihood of the model: this term rewards model that better fits the data

it is not an absolute score, it can be used only to compare models fitted using the same dataset

Bayesian Information Criterion (BIC)

BIC is a criterion for model selection that balances model fit and model complexity. It is similar in purpose to AIC but imposes a stronger penalty for additional parameters, especially as the sample size increases.

number of samples

$$BIC = kln(m) - 2\ln(L)$$

number of parameters of the model: this terms penalizes complex models with a high number of parameters

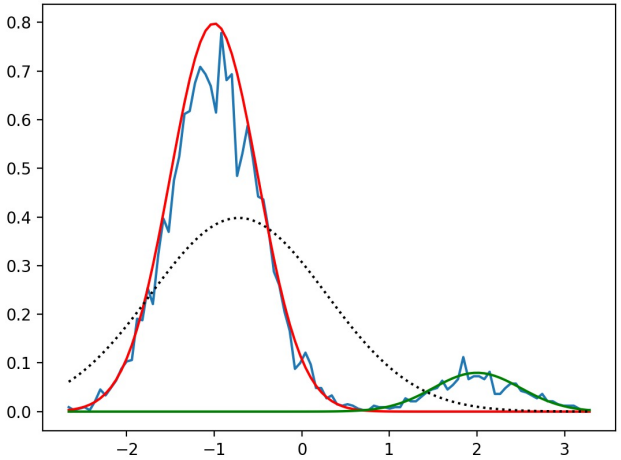
maximum likelihood of the model: this term rewards model that better fits the data

AIC/BIC for the definition of the optimal number of clusters

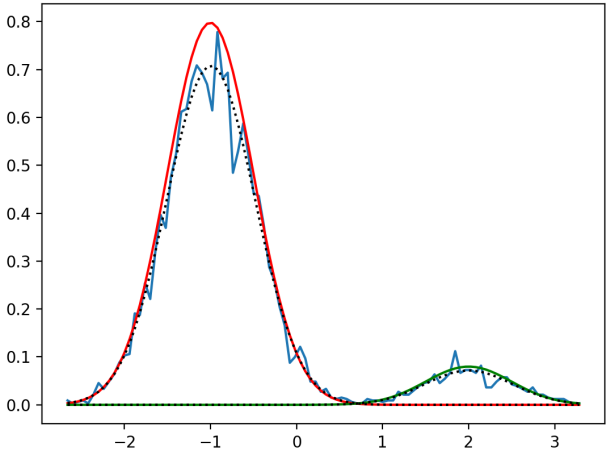
- Define K as the value that minimize AIC/BIC
- It can be used when it's possible to compute a likelihood function (so for example in GMM)

$$L_K = \sum_{i=0}^{m-1} ln\left(\sum_{k=0}^{K} \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)\right)$$

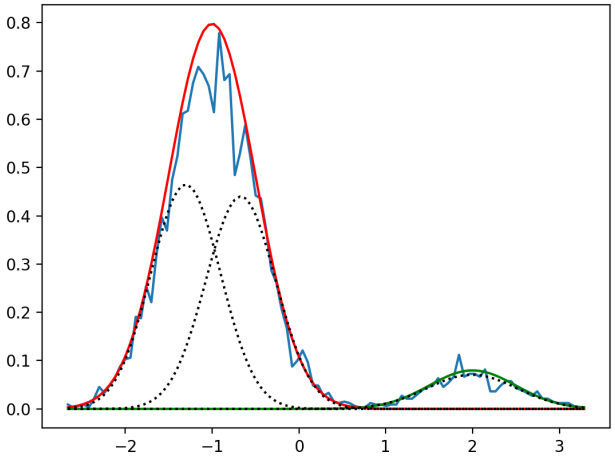
k = 1 AIC = 15581 BIC = 15594



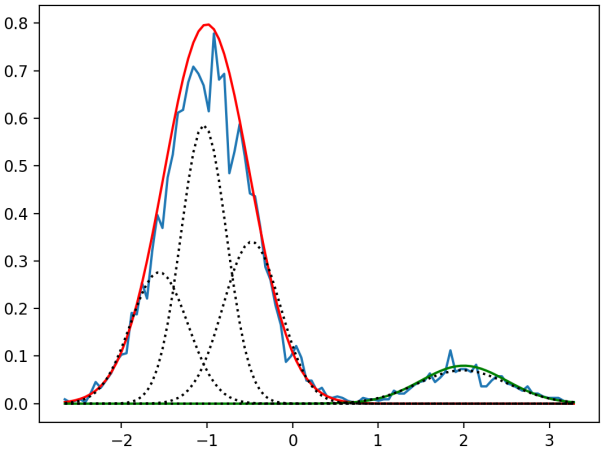
k = 2 AIC = 11206 BIC = 11239



k = 3 AIC = 12240 BIC = 11292

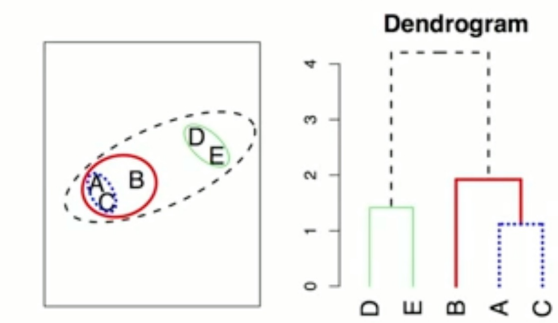


k = 4 AIC = 11237 BIC = 11310



Hierarchical Clustering (bottom-up)

- Algorithm:
  1. Search for the closest set of samples
  2. Merge them
  3. Go back to 1
- It does not require to define the number of clusters
- It produces a dendrogram
- How do we measure the distance between clusters ?
  - average = measure the distance between each pair and take the average
  - single = measure the distance between the closest pair
  - complete = measure the distance between the furthest pair
  - centroid = measure the distance between centroid
- Computational cost scales as N^2



DBSCAN = Density Based Spatial Clustering with Noise

- It searches for regions of high density
- It requires two input parameters (that indirectly define the number of clusters)
  - epsilon = it's a distance, used to define which samples are considered neighbours
  - M = it's a number a points, used to define the core points
- Algorithm
  - For each sample find the points closer than epsilon (nearest neighbours)
  - Define as a core point each point with more than M nearest neighbours
  - pick a random core point (still not assigned to a cluster) and assign it to cluster
    - assign to the cluster all the points in the neighbourhood
    - it a core point is added to the cluster go back repeat previous step on that point
  - Points not assigned to clusters are labelled as noise

Self Organizing Maps

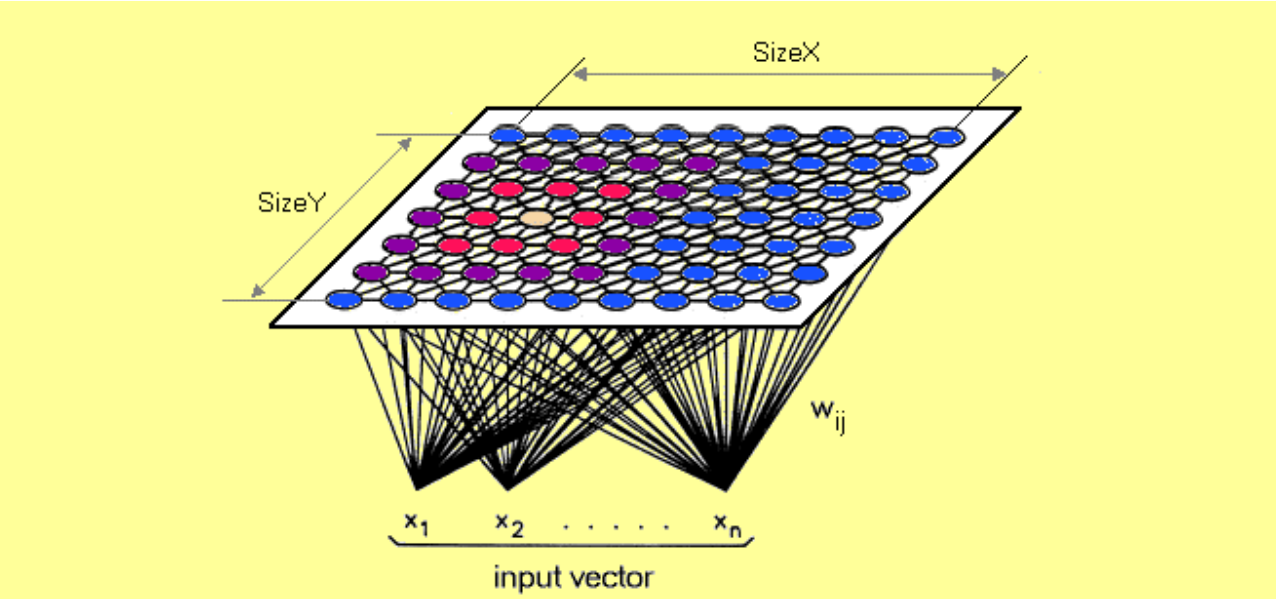
- A Self-Organizing Map is a type of artificial neural network that:
  - Learns to represent input data in a lower-dimensional (usually 2D) discrete lattice
  - Preserves the topological relationships of the input space
- It can be described as a set of neurons arranged on a grid (2D), where each neuron has an associated weight vector of the same dimension as the input date
- Whan an input vector is presented, the algorithm finds the neuron whose weights vector is closest to the input (usually Euclidean distance). This neuron is called the Best Matching Unit (BMU).
- The BMU and it's neighbouring neurons are updated:

$$w_i(t+1) = w_i(t) + \alpha(t) h_{BMU}(i)(x - w_i(t))$$

learning rate (decrease over time)

decreasing function of the distance from the BMU (gaussian)

- Repeating the previous two operations, the neurons specialize to represent regions of space, with nearby neurons representing similar patterns.



https://medium.com/@abhinavr8/self-organizing-maps-ff5853a118d4

https://upload.wikimedia.org/wikipedia/commons/3/35/TrainSOM.gif

Once trained the network can be used to cluster data

- assign a point to the BMU
- use the weights of the network as inputs to a clustering algorithms (as K-means) and then assign the sample to the cluster of its BMU

The second option is more common because with the 1st option the number of clusters is usually too high. Clustering using the weigths is justified because the SOM preserves the topology

Common similarity metrics

- Euclidean distance (L2 norm) $d(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$
- Manhattan distance (L1 norm) $d(x, y) = \|x - y\|_1 = \sum_{i=1}^{n} |x_i - y_i|$
- Cosine similarity $d(x, y) = \frac{x^T y}{\|x\|_2 \|y\|_2} = \cos(\theta)$
  - with teta angle between the vectors x and y

Why Manhattan distance is useful

- Performs well when features represents independent additive quantities
- L1 is less sensible than L2 to outliers (it grows linearly not quadratically)
- In sparse datasets it measures the total coordinate-wise distance

Why cosine similarity is useful

- In high dimensional space the relative Euclidean distance becomes less informative
- Angles can still discriminate points if some structure exists in the high dimensional space

Adjusted Rand Index

How often are two samples paired together compared to what expected by chance

- The Adjusted Rand Index (ARI) measures how similar two clusterings schems (y_true, y_pred) are, correcting for agreement that could occur by chance
- For each pair of samples there are 4 possibilities:
  - case a = the two samples are in the same cluster both in y_true and y_pred
  - case b = the two samples are in the different clusters both in y_true and y_pred
  - case c = the two samples are in the same cluster in y_true and in different clusters in y_pred
  - case d = the two samples are in the same cluster in y_pred and in different clusters in y_true
- The Rand Index, RI, is defined as (#a + #b) / (#a + #b + #c + #d). It measure the proportion of agreements

$$ARI = \frac{RI - \langle RI \rangle}{\max(RI) - \langle RI \rangle}$$

expected RI for random clustering

value of RI when y_true and y_pred are the same

- ARI = 1 → y_true and y_pred are identical
- ARI = 0 → random (dis)agreement